



CS2023 - Programación III

Proyecto: **Pong AI**

Rubén Rivas Medina

Mayo 2025

Descripción general

Desarrollar un agente de IA para jugar Pong usando C++.

Resumen de Epics

- Epic 1: Biblioteca Genérica de Álgebra
- Epic 2: Red Neuronal Full
- Epic 3: Agente Pong basado en la Red
- Epic 4: Paralelismo y CUDA Opcional
- Epic 5: Entrenamiento, Validación y Documentación

Organización y Evaluación

Apendice: Lógica Interna de PongAgent

Epic 1: Biblioteca Genérica de Álgebra

Objetivo: Implementar `Tensor<T, Rank>` en C++20 para arrays multidimensionales estilo NumPy con un único método de acceso variádico.

Interfaz de `Tensor<T, Rank>`:

Namespace:

```
utec::algebra
```

Constructores:

```
Tensor(const std::array<size_t, Rank>& shape);  
template<typename... Dims> Tensor(Dims... dims); // sizeof...(Dims) == Rank
```

Acceso variádico (único):

```
template<typename... Idxs> T& operator()(Idxs... idxs);  
template<typename... Idxs> const T& operator()(Idxs... idxs) const;
```

Información de dimensiones:

```
const std::array<size_t, Rank>& shape() const noexcept;  
void reshape(const std::array<size_t, Rank>& new_shape);  
// throw std::invalid_argument si cambia el número total de elementos
```

Modificación masiva:

```
void fill(const T& value) noexcept;
```

Operadores aritméticos:

```
Tensor operator+(const Tensor& other) const;  
Tensor operator-(const Tensor& other) const;  
Tensor operator*(const Tensor& other) const; // broadcasting en dim de tamaño 1  
Tensor operator*(const T& scalar) const;
```

Utilidades (solo para Rank=2):

```
Tensor transpose_2d() const;
```

Casos de prueba:

Caso 1: Creación, acceso y fill

```
Tensor<int, 2> t(2, 3);  
t.fill(7);  
int x = t(1, 2);
```

Salida esperada:

```
x == 7
```

Caso 2: Reshape válido y acceso lineal

```
Tensor<int,2> t2(2,3);  
t2.reshape(3,2);  
int y = t2[5];
```

Salida esperada:

```
y == t2(2,1) // último elemento después del reshape
```

Caso 3: Reshape inválido

```
Tensor<int,3> t3(2,2,2);  
t3.reshape(2,4,1);
```

Salida esperada:

```
std::invalid_argument thrown
```

Caso 4: Suma y resta de tensores

```
Tensor<double,2> a(2,2), b(2,2);  
a(0,1) = 5.5;  
b.fill(2.0);  
auto sum = a + b;  
auto diff = sum - b;
```

Salida esperada:

```
sum(0,1) == 7.5  
diff(0,1) == 5.5
```

Caso 5: Multiplicación escalar y tensores 3D

```
Tensor<float,1> v(3);  
v.fill(2.0f);  
auto scaled = v * 4.0f;  
Tensor<int,3> cube(2,2,2);  
cube.fill(1);  
auto cube2 = cube * cube;
```

Salida esperada:

```
scaled(2) == 8.0f  
cube2(1,1,1) == 1
```

Caso 6: Broadcasting implícito

```
Tensor<int,2> m(2,1);  
m(0,0)=3; m(1,0)=4;  
Tensor<int,2> n(2,3);  
n.fill(5);  
auto p = m * n;
```

Salida esperada:

```
p(0,2) == 15  
p(1,1) == 20
```

Caso 7: Transpose 2D

```
Tensor<int,2> m2(2,3);  
auto mt = m2.transpose_2d();
```

Salida esperada:

```
mt.shape() == std::array<size_t,2>{3,2}  
mt(0,1) == m2(1,0)
```

Epic 2: Red Neuronal Full

Objetivo: Crear un framework en C++20 que implemente una red neuronal end-to-end usando únicamente para todos los cálculos de **forward** y **backward**, integración de funciones de pérdida y optimización por gradiente descendente.

Interfaz de Red Neuronal:

- Namespace:

```
utec::neural_network
```

- layer.h

```
namespace utec::neural_network {
    template<typename T>
    class ILayer {
    public:
        virtual ~ILayer() = default;
        // Forward: recibe batch x features, devuelve batch x units
        virtual Tensor<T,2> forward(const Tensor<T,2>& x) = 0;
        // Backward: recibe gradiente de salida, devuelve gradiente de entrada
        virtual Tensor<T,2> backward(const Tensor<T,2>& grad) = 0;
    };
}
```

- dense.h

```
#include "utec/algebra/tensor.h"
namespace utec::neural_network {
    template<typename T>
    class Dense : public ILayer<T> {
        Tensor<T,2> W, dW; // [in, out] y su gradiente
        Tensor<T,1> b, db; // [out] y su gradiente
        Tensor<T,2> last_x; // cache de entrada para backward

    public:
        Dense(size_t in_feats, size_t out_feats);
        Tensor<T,2> forward(const Tensor<T,2>& x) override;
        Tensor<T,2> backward(const Tensor<T,2>& grad) override;
    };
}
```

- activation.h

```
namespace utec::neural_network {
    template<typename T>
    class ReLU : public ILayer<T> {
        Tensor<T,2> mask; // para almacenar dónde x>0
    public:
        Tensor<T,2> forward(const Tensor<T,2>& x) override;
        Tensor<T,2> backward(const Tensor<T,2>& grad) override;
    };
}
```

- loss.h

```

namespace utec::neural_network {
    template<typename T>
    class MSELoss {
        Tensor<T,2> last_pred, last_target;
    public:
        // Devuelve la pérdida media
        T forward(const Tensor<T,2>& pred, const Tensor<T,2>& target);
        // Devuelve dL/dpred
        Tensor<T,2> backward();
    };
}

```

Clase NeuralNetwork

```

#include "layer.h"
#include <vector>
namespace utec::neural_network {
    template<typename T>
    class NeuralNetwork {
        std::vector<std::unique_ptr<ILayer<T>>> layers;
        MSELoss<T> criterion;
    public:
        void add_layer(std::unique_ptr<ILayer<T>> layer);
        // Ejecuta forward por todas las capas
        Tensor<T,2> forward(const Tensor<T,2>& x);
        // Lanza backward desde la última capa
        void backward(const Tensor<T,2>& grad);
        // Actualiza todos los parámetros con learning rate lr
        void optimize(T lr);
        // Entrena con X, Y durante epochs
        void train(const Tensor<T,2>& X, const Tensor<T,2>& Y, size_t epochs, T lr);
    };
}

```

Casos de prueba

Ejemplo de prueba: ReLU forward/backward

```

#include "utec/algebra/tensor.h"
#include "utec/nn/activation.h"
using T = float;
Tensor<T,2> M({2,2});
M(0,0) = -1; M(0,1) = 2;
M(1,0) = 0; M(1,1) = -3;

auto relu = utec::nn::ReLU<T>();
auto R = relu.forward(M);
std::cout << R(0,1) << "\n"; // espera 2

Tensor<T,2> GR({2,2}); GR.fill(1.0f);
auto dM = relu.backward(GR);

std::cout << dM(0,0) << "," << dM(1,1);

```

Ejemplo de prueba: MSELoss forward/backward

```
#include "utec/algebra/tensor.h"
#include "utec/nn/loss.h"
using T = double;
Tensor<T,2> P({1,2}); P(0,0)=1; P(0,1)=2;
Tensor<T,2> Tgt({1,2}); Tgt(0,0)=0; Tgt(0,1)=4;
auto loss = utec::nn::MSELoss<T>();
T L = loss.forward(P, Tgt);
std::cout << L << "\n"; // espera 2.5
Tensor<T,2> dP = loss.backward();
std::cout << dP(0,1);
```

Ejemplo de prueba: Entrenamiento XOR

```
#include "utec/algebra/tensor.h"
#include "utec/nn/neural_network.h"
using T = float;
// Datos XOR
Tensor<T,2> X({4,2}); // {{0,0},{0,1},{1,0},{1,1}}
Tensor<T,2> Y({4,1}); // {{0},{1},{1},{0}}

utec::nn::NeuralNetwork<T> net;
net.add_layer(std::make_unique<utec::nn::Dense<T>>(2,4));
net.add_layer(std::make_unique<utec::nn::ReLU<T>>());
net.add_layer(std::make_unique<utec::nn::Dense<T>>(4,1));

float final_loss = net.train(X, Y, 1000, 0.1f);
std::cout << (final_loss < 0.1f);
```

Ejemplo de prueba: Shape mismatch

```
#include "utec/algebra/tensor.h"
#include "utec/nn/neural_network.h"
using T = float;
utec::nn::NeuralNetwork<T> net;
// Asumiendo que ya has añadido capas al net
try {
    net.forward(Tensor<T,2>({3,3}));
} catch (const std::invalid_argument&) {
    std::cout << "exception";
}
```


Epic 3: Agente Pong basado en la Red

Objetivo. Desarrollar en C++ una clase PongAgent que, usando el modelo entrenado, interactúe con un entorno simplificado (EnvGym) para jugar Pong. El agente deberá:

- Recibir el estado actual del juego (State) con posición de bola y paleta.
- Decidir acción (-1=arriba, 0=quieto, +1=abajo) mediante un único método act().
- Mantener un bucle de simulación que invoque forward de la red para cada nuevo estado.

Interfaces principales

State: define el estado del juego.

```
namespace utec::nn {
    struct State {
        float ball_x, ball_y;
        float paddle_y;
    };
}
```

EnvGym: entorno mínimo.

```
namespace utec::nn {
    class EnvGym {
    public:
        State reset();
        State step(int action, float &reward, bool &done);
    };
}
```

PongAgent: implementa la lógica de acción.

```
namespace utec::nn {
    template<typename T>
    class PongAgent {
        std::unique_ptr<ILayer<T>> model;
    public:
        PongAgent(std::unique_ptr<ILayer<T>> m);
        // Convierte State a Tensor< T,2 >, llama forward y devuelve -1/0/+1
        int act(const State &s);
    };
}
```

Guía de uso y casos de prueba

1. Instanciación básica

```
using T = float;
// Supón que 'net' es tu NeuralNetwork<T> ya entrenada...
auto agent = utec::nn::PongAgent<T>{
    std::make_unique<utec::nn::Dense<T>>(3,3) // reemplaza con tu modelo
};
// Estado de prueba: bola por encima de paleta
utec::nn::State s{0.5f, 0.8f, 0.3f};
int a = agent.act(s);
std::cout << a << "\n"; // espera +1 (subir)
```

2. Simulación de un paso

```
utec::nn::EnvGym env;  
float reward; bool done;  
auto s0 = env.reset();  
int a0 = agent.act(s0);  
auto s1 = env.step(a0, reward, done);  
std::cout  
    << s1.ball_x << ", "  
    << reward << ", "  
    << done << "\n";
```

3. Integración agent + entorno Ejecuta varios pasos sin aprendizaje:

```
for(int t=0; t<5; ++t) {  
    int a = agent.act(s0);  
    s0 = env.step(a, reward, done);  
    std::cout << "Step " << t  
                << " action=" << a  
                << " reward=" << reward  
                << "\n";  
    if(done) break;  
}
```

4. Prueba de límites Verifica que cuando ball_y == paddle_y, act() devuelva 0:

```
utec::nn::State eq{0.2f,0.5f,0.5f};  
std::cout << (agent.act(eq)==0) << "\n"; // espera 1 (true)
```

Epic 4: Paralelismo y CUDA Opcional

Objetivo. Añadir `ThreadPool` y cola concurrente para procesar inferencias en paralelo; opcionalmente soporte CUDA.

Epic 5: Entrenamiento, Validación y Documentación

Objetivo. Desarrollar pipeline de entrenamiento básico (p.ej. hill-climbing), serializar modelo y documentar.

Organización y Evaluación

Estructura del proyecto:

```
pong_ai/
+-- include/          # cabeceras principales
|   +-- utec/
|       +-- algebra/  # álgebra: implementa Tensor<T,Rank>
|           | +-- Tensor.h
|           +-- nn/   # red neuronal: define capas y NeuralNetwork
|               | +-- layer.h
|               | +-- dense.h
|               | +-- activation.h
|               | +-- loss.h
|               | +-- neural_network.h
|           +-- agent/ # agente y entorno: PongAgent y EnvGym
|               +-- PongAgent.h
|               +-- EnvGym.h
+-- src/              # archivos fuentes (si se requiriera)
|   +-- utec/
|       +-- agent/    # agente y entorno: PongAgent y EnvGym
|           +-- PongAgent.cpp
|           +-- EnvGym.cpp
+-- tests/            # casos de prueba automatizados para Gradescope
|   +-- test_tensor.cpp
|   +-- test_neural_network.cpp
|   +-- test_agent_env.cpp
+-- docs/             # documentación del proyecto y bibliografía
|   +-- README.md
|   +-- BIBLIOGRAFIA.md
```

Reglas de formación de grupos:

- Cada grupo de 4 a 5 alumnos (mínimo 4).
- Todos los alumnos individual y obligatoriamente deben desarrollar los Epics #1 y #2.
- El Epic #3 se puede hacer de forma individual o grupal; el grupo selecciona la mejor implementación y la integra con los mejores Epics #1/#2 del grupo.

Evaluación:

- Si el Epic #3 funciona correctamente, el grupo obtiene automáticamente la nota 12.
- Fixture de competencia entre grupos en 4 niveles:
 1. Nivel 1: ganadores → 14, perdedores → 13.
 2. Nivel 2: ganadores → 16, perdedores → 15.
 3. Nivel 3: ganadores → 18, perdedores → 17.
 4. Nivel 4: ganadores → 20, perdedores → 19.
- Si el Epic #3 funciona correctamente, el grupo obtiene automáticamente la nota 12.

Apéndice: Lógica Interna de PongAgent

El propósito de este PongAgent es, dado el estado del juego, decidir mover la paleta (-1, 0, +1) y aprender de la experiencia.

Cálculo de la recompensa

La recompensa (float reward) se define, por ejemplo, así:

- +1 si el agente devuelve la bola exitosamente.
- -1 si falla y pierde el punto.
- 0 en cualquier otro paso.

Se recibe tras cada llamada a:

```
State next_state;
float reward;
bool done;
next_state = env.step(action, reward, done);
```

Modalidades de entrenamiento

On-policy: Actualiza la política inmediatamente con la experiencia reciente (e.g., SARSA):

```
// Tras env.step:
Tensor<float,2> Q_pred = net.forward(x_t);
Tensor<float,2> Q_next = net.forward(x_{t+1});
float target = r + gamma * Q_next(0, a_{t+1});
net.backward(Q_pred - target);
net.optimize(lr);
```

Off-policy: Almacena (s,a,r,s',done) en un replay buffer y entrena por lotes (e.g., DQN):

```
buffer.push({s,a,r,s_next,done});
auto batch = buffer.sample(batch_size);
// Calcular targets con max_a' Q(s',a')
net.backward(batch.Q_pred - batch.Q_target);
net.optimize(lr);
```

Bucle de entrenamiento completo

```
for(int epoch = 1; epoch <= N; ++epoch) {
    State s = env.reset();
    bool done = false;
    while(!done) {
        int a = agent.act(s);
        State s_next;
        float r;
        s_next = env.step(a, r, done);
        // On-policy (o almacenarlo para off-policy):
        agent.learnOnPolicy(s, a, r, s_next);
        // Off-policy (opcional):
        // agent.store(s, a, r, s_next, done);
        // agent.learnOffPolicy();
        s = s_next;
    }
}
```

Aquí, el agente alterna entre explorar y explotar, ajustando sus pesos para maximizar la recompensa acumulada (win-rate).